

Git for Developers

Closing — Pitfalls, Cheat Sheet & Resources

Common Mistakes, Quick Reference, and What to Learn Next

Closing — Agenda

- Common pitfalls and how to avoid them
- Git command cheat sheet
- Recommended resources and next steps
- Q&A

Part 1 — Common Pitfalls

Pitfall 1 — Committing Secrets

```
# Someone accidentally committed an API key
git commit -m "add env config"
# → .env file with SECRET_KEY=sk-live-abc123 is now in history

# Even after deleting the file, it's still in history!
git rm .env
git commit -m "remove env"
# The secret is still in git log -p HEAD~1
```

How to avoid it

```
# Add secrets to .gitignore BEFORE the first commit
echo ".env" >> .gitignore

# If already committed: use git-filter-repo to purge
pip install git-filter-repo
git filter-repo --path .env --invert-paths

# Rotate the secret immediately – it must be considered compromised
```

Pitfall 2 — Committing to the Wrong Branch

```
# You committed 3 things directly to main by accident
git log --oneline -3
# a3f7c21 feat: payment integration
# b2e1d40 feat: shipping calculator
# c9a8b12 feat: tax module

# Move those commits to a feature branch
git branch feature/ecommerce          # create branch at current HEAD
git reset --hard origin/main          # move main back to origin
git checkout feature/ecommerce         # the 3 commits are still here
```

Pitfall 3 — Force-Pushing Shared Branches

```
# Dangerous — overwrites others' work  
git push --force origin main
```

```
# Safer — aborts if someone pushed since your last fetch  
git push --force-with-lease origin main
```

```
# Safest — just don't force-push shared branches  
# Rewrite history only on your local, private feature branches
```

Pitfall 4 — Merge vs. Rebase Confusion

```
# After git pull --rebase, your commits were rewritten
# (new SHAs) — don't panic!

git pull --rebase
# CONFLICT (content): Merge conflict in src/auth.js

# Resolve, then:
git add src/auth.js
git rebase --continue      # NOT git commit!

# Abort if you change your mind
git rebase --abort
```


Pitfall 5 — Losing Work in `git reset --hard`

```
# You ran: git reset --hard HEAD~5 and lost 5 commits
# Recovery via reflog
git reflog
# a3f7c21 HEAD@{6}: commit: feat: last important commit

git reset --hard a3f7c21    # you're back!
```

The reflog saves you from almost everything — except `git clean -fdx` (irreversible).

Pitfall 6 — Giant Commits / PRs

- ✗ One commit: "Implement entire e-commerce module"
→ impossible to review, impossible to bisect, impossible to revert cleanly
- ✓ Many small commits:
 - feat: add product catalog data model
 - feat: add product listing API endpoint
 - feat: add cart service
 - feat: add checkout flow
 - feat: add payment gateway integration

Part 2 — Git Cheat Sheet

Setup & Configuration

```
git config --global user.name "Jane Smith"  
git config --global user.email "jane@example.com"  
git config --global core.editor "code --wait"  
git config --global init.defaultBranch main  
git config --list
```

Creating & Cloning

```
git init                                # initialize a new repository
git clone <url>                          # clone a remote repository
git clone --depth 1 <url>                # shallow clone (CI-friendly)
git clone --recurse-submodules <url>    # clone with submodules
```

Daily Workflow

```
git status          # what's changed?
git add <file>      # stage a file
git add .           # stage everything
git add -p          # stage interactively (hunk by hunk)
git commit -m "feat: ..." # commit with message
git commit --amend   # fix last commit (before push)
git push            # push to remote
git pull --rebase    # pull and rebase
```

Branching

```
git branch                # list local branches
git branch -a             # list all branches (+ remote)
git checkout -b feature/x # create and switch
git switch -c feature/x   # modern equivalent
git checkout main         # switch to main
git merge feature/x       # merge into current branch
git merge --no-ff feature/x # merge with merge commit
git branch -d feature/x   # delete merged branch
git branch -D feature/x   # force-delete
```

History & Inspection

```
git log --oneline --graph --all # visual branch history
git log --grep="auth"          # search commit messages
git diff                       # unstaged changes
git diff --staged              # staged changes
git show <hash>                # show a commit
git blame <file>               # annotate file with commit info
```


Undoing

```
git restore <file>           # discard unstaged changes
git restore --staged <file>  # unstage a file
git reset --soft HEAD~1      # undo commit, keep staged
git reset --hard HEAD~1      # undo commit, discard changes
git revert <hash>             # safe undo (creates new commit)
git reflog                   # find lost commits
```

Advanced

```
git rebase main                # update branch with main's changes
git rebase -i HEAD~4          # interactive rebase
git cherry-pick <hash>        # apply one commit to current branch
git stash                     # save WIP
git stash pop                 # restore WIP
git tag -a v1.0.0 -m "... "   # create annotated tag
git bisect start/good/bad     # binary search for a bug
git clean -n                  # preview removal of untracked files
git clean -fd                 # remove untracked files + dirs
```

Part 3 — Resources & Next Steps

Essential Reading

Resource	URL / Description
Pro Git (book)	<code>git-scm.com/book</code> — free online, comprehensive
Official Git docs	<code>git-scm.com/docs</code> — every command explained
Conventional Commits	<code>conventionalcommits.org</code>
Gitflow	<code>nvie.com/posts/a-successful-git-branching-model</code>
GitHub Docs	<code>docs.github.com</code>
GitLab Docs	<code>docs.gitlab.com</code>

Interactive Learning

Platform	Content
Learn Git Branching	<code>learngitbranching.js.org</code> — visual, interactive branching exercises
GitHub Skills	<code>skills.github.com</code> — guided repo-based courses
Katacoda / Killercoda	Hands-on terminal scenarios
Oh My Git!	Game-based Git learning

Tools to Explore Next

Tool	Purpose
Husky	Git hooks for Node.js projects
commitlint	Enforce Conventional Commits in CI
git-filter-repo	History rewriting (safer than <code>filter-branch</code>)
BFG Repo Cleaner	Remove large files or secrets from history
Scalar / VFS for Git	Scale Git for very large repos
Nx / Turborepo	Monorepo tooling
lazygit	Terminal UI for Git

Thank You!

Key Takeaways

- **Commit small, commit often** — atomic commits make everything easier
- **Branch freely** — they're cheap and safe
- **Never rewrite shared history** — use `git revert` on public branches
- **The reflog is your safety net** — most "lost" work is recoverable
- **Automate with hooks and CI** — let the machine enforce quality

Q&A

"The most powerful Git skill is knowing when NOT to use a complex command — and choosing the simple thing that works."

Keep Practising

```
# Today's homework
git init my-practice-repo
cd my-practice-repo
git checkout -b feature/experiment
# ... make some commits, try a rebase, resolve a conflict ...
git log --oneline --graph --all
```

Good luck, and happy committing!